

Productization Roadmap & Decision Log

Owner: Alok (solo) · **Last updated:** 2026-07-07

Purpose: Capture the plan and key decisions for turning this single-pharmacy app into a commercial product sold to medical store owners — so future-me can recall *what* was decided and *why*, without re-deriving it.

1. The two-version strategy

Version	Git branch	What it is
Personal	phase1-customers	The live app running my own store (Fahrenheit Pharmacy). Stable, deployed as-is. Touch only for real fixes.
Commercial	commercial	Where all productization work happens. Same baseline today; will diverge as multi-tenancy etc. is built.

Guiding principle: diverge now, converge later. Multi-tenancy is a *superset*, not a different product. The end state is **one codebase where my own store is "tenant #1"** (dogfooding). Once the commercial version is solid, migrate personal onto it and retire the personal fork.

Avoid the solo-dev trap: don't let the two branches rot apart. Fix shared bugs on `phase1-customers`, then merge into `commercial`. Only split into a separate repo once the versions have genuinely diverged (auth, billing, multi-tenancy).

2. Current architecture reality (2026-07-07)

- **Single-tenant.** No `tenant_id` / store / org concept anywhere. All sales, customers, credit, returns live in **one shared Supabase project** (`pharmacylocal`).
- **Pharmacy identity is in the browser's `localStorage`** (`usePharmacyProfile`), hardcoded default with a real GSTIN/DL — not in the DB, not per-account.
- **Customers** are a first-class entity (Phase 1) with `customer_id` linkage.
- **Medicines** are **free-text** on each sale, cleaned by canonicalize/dedup (`src/utils/medicine.ts`); autocomplete derives from distinct names in `sales`. **There is no medicines master table.**
- **Returns** exist (`sales_returns` + credit-note logic), visible in the UI.

Implication: onboarding a second store today would let it see the first store's data. That is the #1 thing to fix before selling.

3. Commercial readiness — priorities (blockers first)

#	Item	Why it matters	Size
1	Multi-tenancy / data isolation	Without it, stores see each other's data — a dealbreaker. RLS by <code>tenant_id</code> on every table; per-tenant pharmacy profile in the DB; tenant-scoped auth.	Large
2	Infrastructure & data safety	Free tier won't do commercially (cold-starts; pauses when idle). Each store needs paid, reliable compute + backups. Data loss = legal liability once someone pays.	Medium
3	Compliance & polish	GST invoice, Credit Note on returns, GST summary reports, schedule-H handling. Where the printed Credit Note earns its keep.	Medium
4	Onboarding & support	Solo operator supporting N customers — self-serve onboarding, docs, update path.	Ongoing
5	Legal / liability	Selling medical billing software carries responsibility — wrong GST or lost data is now <i>my</i> liability. Robustness + testing matter more.	Ongoing

Interim way to "go professional" now (low risk)

Start with **1–3 pilot stores, each on their own isolated deployment** (separate Supabase project + separate hosted build). Data is naturally isolated — **zero multi-tenancy code needed** — so I can validate willingness to pay *before* building heavy multi-tenancy.

Suggested build order for the commercial branch

1. Multi-tenancy foundation (`tenant_id` + RLS everywhere; pharmacy profile → DB).
2. **Medicines master table + inventory/stock** (see Decision D-1).
3. Compliance (proper Credit Note with numbering, GST reports).
4. Billing/subscription + onboarding.
5. Paid infra + backups **before the first paying (non-pilot) customer**.

4. Decision log

D-1 — Medicines master table: DEFER to the commercial version (2026-07-07)

Decision: Do **not** build a `medicines` master table (catalog + `medicine_id`) in the personal version now. Build it as an early foundational piece of the commercial version, together with multi-tenancy and inventory.

Why:

- Personal works; the immediate pain (duplicate/misspelled names) is **already solved** by canonicalize-on-blur + dedup.
- A `medicine_id` backfill across thousands of live sales rows is a Phase-1-customers-sized effort with **real risk on the DB that runs my actual business**, for benefits I won't feel yet.
- The real payoff is **commercial-tier**: inventory/stock, reorder levels, HSN/GST per medicine, manufacturer, schedule H/H1, purchase management — deserves deliberate design.
- It would be **partly redone for multi-tenancy anyway** (needs `tenant_id`; per-store vs shared global drug DB is open). Building single-tenant now doesn't avoid that work.
- The "data keeps piling up" worry is **weaker here** than for customers: after canonicalization, names are already consistent, so the eventual backfill is mechanical.

What would flip this: if I decide to add **inventory / stock tracking to my own store soon** (not just sell it), the medicines master becomes the required foundation and worth doing on the shared baseline.

D-2 — Printed Credit Note: DEFER; do the proper version in commercial (2026-07-07)

The on-screen invoice already shows returns (banner + per-line). The *printed* invoice still shows the original amount only. The compliant fix is a **separate Credit Note document** (not editing the original invoice). Low-risk (isolated print function; data already in `sales_returns`); the only moderate part is a unique sequential **CN number** (one additive column + numbering + migration).

- Selling to GST-registered stores → build the proper Credit Note (with CN numbering). Table stakes.
- A lightweight "Return Receipt" print (no schema change) is a valid quick option for personal before then.

Placed as a **step-3 (compliance)** item, not a blocker.

D-3 — Performance optimization: NO CHANGE for now (2026-07-07)

Investigated Dashboard/Sales Register/Reports load time. Numbers are **correct**; only speed. Small ranges ~200ms (network floor, fine); large ranges + Dashboard ~1s (pulling ~1,250 full rows to compute totals). Decided to do nothing for now — see **performance-notes.md**. If revisited: lazy-load Last Month (zero risk) → Dashboard aggregates (shadow-verified) → paid infra. **Do not** "add a `sale_date` index" — it already exists.

D-4 — Two-version strategy via branches (2026-07-07)

Chosen over duplicate repos so shared fixes flow easily between personal and commercial. See §1.

D-5 — WhatsApp receipts: free wa.me in personal, Cloud API as a commercial feature (2026-07-07)

Personal (implemented): a free WhatsApp "click-to-chat" ([wa.me](#)) button on the Sales Register ACTION column (after Print). It opens WhatsApp with a pre-filled receipt message to the customer; the pharmacist presses send. No API, no cost, no ban risk. [src/utils/whatsapp.ts](#) (+ unit tests); wired into [src/pages/SalesList.tsx](#) . *Limitations:* pre-fills **text only** (no PDF attachment) and needs **one manual tap** per invoice.

Commercial (planned, paid) — WhatsApp Cloud API: fully automated sending **and PDF invoice attachment** requires the official **WhatsApp Cloud API** (Meta). Carries a **per-message fee** (utility templates), and needs a **dedicated phone number** (not usable on normal WhatsApp), **business verification**, and **pre-approved templates**. Offer as a **premium add-on** — the automation + PDF business customers expect, a natural upsell over free tap-to-send. 🚫 **Do NOT** use unofficial libraries (whatsapp-web.js / Baileys) — they violate WhatsApp's Terms and risk the number being **banned**.

5. Related documents

- [performance-notes.md](#) — load-time investigation & decision.
- [Customer-Data-Correction-Analysis.pdf](#) — earlier analysis.
- [Strategic-Solution-Customer-Identity-Architecture.pdf](#) — earlier identity architecture.
- [DEPLOY.md](#) — build & deploy steps.

6. Open items / to revisit

- Multi-tenancy design (per-store vs shared drug DB; RLS strategy).
- Medicines master + inventory (D-1) — commercial.
- Proper Credit Note with numbering (D-2) — commercial compliance.
- Pharmacy profile: move from localStorage to per-tenant DB row.
- Paid Supabase tier + automated backups before first paying customer.
- Pricing/licensing model (SaaS subscription vs one-time).
- WhatsApp **Cloud API** auto-send + PDF invoice (D-5) — commercial premium feature.